Assignment No. 07

Assignment based on Model Persistence and Model Serving via Web API

Exemplars: Train a simple classifier (e.g., Logistic Regression) on a dataset like Iris. Save the model using joblib or pickle. Create a simple Flask or Streamlit application with an endpoint that loads the saved model and serves predictions.

Source Code:

1. **train_model.py**

```
import pandas as pd
import pickle

from sklearn.preprocessing import LabelEncoder, StandardScaler
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score
from sklearn.utils import resample

# ==============================
# STEP 1: LOAD DATA
# ==============================
df = pd.read_csv("space_mission_dataset.csv")

# Clean column names
df.columns = df.columns.str.strip()

# ==============================
# STEP 2: PREPROCESSING
# ==============================

# Convert datetime
df['Datetime'] = pd.to_datetime(df['Datetime'], errors='coerce')

# Extract features
df['Year'] = df['Datetime'].dt.year
df['Month'] = df['Datetime'].dt.month

# Select required columns
df = df[['Organisation', 'Location', 'Details', 'Year', 'Month',
'Mission_Status']]

# Drop missing values
df.dropna(inplace=True)

# Convert target variable
df['Mission_Status'] = df['Mission_Status'].apply(
```

```
    lambda x: 1 if 'Success' in str(x) else 0
)

# ==============================
# STEP 3: CHECK IMBALANCE
# ==============================
print("Before Balancing:")
print(df['Mission_Status'].value_counts())

# ==============================
# STEP 4: BALANCE DATA (UPSAMPLING)
# ==============================

df_success = df[df['Mission_Status'] == 1]
df_failure = df[df['Mission_Status'] == 0]

# Upsample failure class
df_failure_upsampled = resample(
    df_failure,
    replace=True,
    n_samples=len(df_success),
    random_state=42
)

# Combine
df_balanced = pd.concat([df_success, df_failure_upsampled])

# Shuffle dataset
df_balanced = df_balanced.sample(frac=1, random_state=42)

df = df_balanced

print("\nAfter Balancing:")
print(df['Mission_Status'].value_counts())

# ==============================
# STEP 5: ENCODING
# ==============================

le_org = LabelEncoder()
le_loc = LabelEncoder()
le_rocket = LabelEncoder()

df['Organisation'] = le_org.fit_transform(df['Organisation'])
df['Location'] = le_loc.fit_transform(df['Location'])
df['Details'] = le_rocket.fit_transform(df['Details'])

# ==============================
```

```
# STEP 6: FEATURES & TARGET
# ============================

X = df[['Organisation', 'Location', 'Details', 'Year', 'Month']]
y = df['Mission_Status']

# ============================
# STEP 7: TRAIN TEST SPLIT
# ============================

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# ============================
# STEP 8: SCALING
# ============================

scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

# ============================
# STEP 9: MODEL TRAINING
# ============================

model = RandomForestClassifier(
    n_estimators=150,
    max_depth=10,
    random_state=42,
    class_weight='balanced'
)

model.fit(X_train, y_train)

# ============================
# STEP 10: EVALUATION
# ============================

y_pred = model.predict(X_test)
accuracy = accuracy_score(y_test, y_pred)

print("\nModel Accuracy:", accuracy)

# ============================
# STEP 11: SAVE MODEL
# ============================
```

```
pickle.dump(model, open("model.pkl", "wb"))
pickle.dump(scaler, open("scaler.pkl", "wb"))
pickle.dump(le_org, open("le_org.pkl", "wb"))
pickle.dump(le_loc, open("le_loc.pkl", "wb"))
pickle.dump(le_rocket, open("le_rocket.pkl", "wb"))

print("\nModel and encoders saved successfully!")
```

2. **app.py**

```
import streamlit as st
import pickle
import numpy as np

# Load saved files
model = pickle.load(open("model.pkl", "rb"))
scaler = pickle.load(open("scaler.pkl", "rb"))
le_org = pickle.load(open("le_org.pkl", "rb"))
le_loc = pickle.load(open("le_loc.pkl", "rb"))
le_rocket = pickle.load(open("le_rocket.pkl", "rb"))

# Title
st.title("🚀 Space Mission Success Prediction")

# User Inputs
organisation = st.selectbox("Select Organisation", le_org.classes_)
location = st.selectbox("Select Location", le_loc.classes_)
rocket = st.selectbox("Select Rocket (Details)", le_rocket.classes_)

year = st.slider("Select Year", 1957, 2030)
month = st.slider("Select Month", 1, 12)

# Prediction Button
if st.button("Predict"):

    # Encode inputs
    org_encoded = le_org.transform([organisation])[0]
    loc_encoded = le_loc.transform([location])[0]
    rocket_encoded = le_rocket.transform([rocket])[0]

    # Prepare input data
    input_data = np.array([[org_encoded, loc_encoded, rocket_encoded, year,
month]])

    # Scale input
    input_data = scaler.transform(input_data)

    # Predict
```

```
prediction = model.predict(input_data)

# Output
if prediction[0] == 1:
    st.success("🚀 Mission Status: SUCCESS")
else:
    st.error("❌ Mission Status: FAILURE")
```

Output: